

FATIMA JINNAH WOMEN UNIVERSITY



DIGITAL LOGIC AND DESIGN **PROJECT REPORT** **BCS-(II-B)**

SUBMITTED TO: Sir Majid Shafique

SUBMITTED BY: Ayman Un Nisa (023)

Maryam Zulfiqar(063)

Farwa Batool(029)

DEPARTMENT OF COMPUTER SCIENCE

FATIMA JINNAH WOMEN UNIVERSITY

RAWALPINDI

www.fjwu.edu.pk

EXPLANATION OF COMPONENTS:

Arduino Uno

1. **Microcontroller:** The heart of the Arduino Uno is a microcontroller, which is a small computer on a single integrated circuit.
2. **Power:** It can be powered via USB from your computer or through a power adapter.
3. **Pins:** It has digital and analog pins that you can use to connect sensors, LEDs, and other components.
4. **Programming:** You write code in the Arduino IDE (Integrated Development Environment) and upload it to the Arduino to make it perform tasks.
5. **Versatility:** It's widely used in DIY projects, prototyping, and educational purposes due to its simplicity and flexibility.

Water Level Sensor

1. **Functionality:** It consists of a series of parallel wires that detect the presence of water at different levels.
2. **Analog Output:** The sensor provides an analog output that varies depending on the water level.
3. **Use Case:** Often used in applications like water tanks, reservoirs, and aquariums to monitor water levels.
4. **Signal:** Sends the water level information as a signal to the Arduino for processing.
5. **Integration:** Easily integrated with other sensors and components in a monitoring system.

Buzzer

1. **Types:** Comes in two types: active (which buzzes on its own) and passive (which needs an external signal to buzz).
2. **Sound Production:** Produces sound through the vibration of a diaphragm when an electrical signal is applied.
3. **Alert Mechanism:** Commonly used in alarms, alerts, and notification systems.
4. **Simple Interface:** Requires just a simple on/off signal from the Arduino to operate.
5. **Power:** Typically operates at low voltage, making it suitable for battery-powered projects.

LEDs (Light Emitting Diodes)

1. **Lighting:** LEDs emit light when current flows through them, available in various colors.
2. **Indication:** Used to indicate status, such as power on/off, error states, or other conditions in a circuit.

3. **Polarity:** Has a positive (anode) and a negative (cathode) side, and only lights up when connected correctly.
4. **Energy Efficiency:** Consumes very little power, making them energy-efficient indicators.
5. **Control:** Can be turned on or off and controlled in brightness using Pulse Width Modulation (PWM) from the Arduino.

Breadboard

1. **Prototyping Tool:** Used for constructing and testing circuits quickly without soldering.
2. **Holes and Rows:** Has many holes arranged in rows and columns for inserting component leads and jumper wires.
3. **Ease of Use:** Components and wires can be easily inserted and removed, allowing for rapid modifications.
4. **Connection Strips:** Contains metal strips under the holes that connect in specific patterns to create circuits.
5. **Reusable:** Can be used over and over again for different projects, making it a staple in electronics prototyping.

Male to Male Jumper Wires

1. **Connections:** Used to connect different points on a breadboard or to connect a breadboard to the Arduino.
2. **Flexibility:** Allows for flexible and temporary connections in a circuit.
3. **Variety:** Available in different lengths and colors for easy identification and organization.
4. **Reusability:** Can be used multiple times in various projects.
5. **Convenience:** Simplifies the process of building and modifying circuits without soldering.

Male to Female Jumper Wires

1. **Versatile Connections:** Connects components with pins (like sensors) to breadboards or other parts of a circuit.
2. **Adaptability:** Useful for connecting to components that have male header pins.
3. **Color Coding:** Typically available in various colors for easy circuit organization.
4. **Reusable:** Can be used repeatedly in different projects.
5. **Length Options:** Comes in different lengths to suit various project needs.

ROLE AND FUNCTION OF EACH COMPONENT IN PROJECT:

In a water level sensor project using an Arduino, each component plays a specific role to detect, indicate, and alert based on the water level. Here's how each component works:

1. Arduino Uno

- **Role:** Acts as the central controller.
- **Function:** Reads input from the water level sensor, processes the data, and controls the output devices (LEDs and buzzer) based on the sensor readings.

2. Water Level Sensor

- **Role:** Detects the water level.
- **Function:** Consists of conductive traces that create multiple parallel lines. When the water touches these lines, it completes circuits that measure the water level by varying the resistance. The sensor outputs an analog signal proportional to the water level.

3. Buzzer

- **Role:** Provides an audible alert.
- **Function:** When activated by the Arduino, it emits a sound to indicate a high water level or another alert condition. The buzzer is connected to a digital output pin on the Arduino.

4. LEDs (3 LEDs)

- **Role:** Provide visual indication of water levels.
- **Function:**
 - **LED 1 (Low level):** Indicates a low water level when lit.
 - **LED 2 (Medium level):** Indicates a medium water level when lit.
 - **LED 3 (High level):** Indicates a high water level when lit.
- **Connection:** Each LED is connected to a digital output pin on the Arduino. When the Arduino sends a HIGH signal to a pin, the corresponding LED lights up.

5. Breadboard

- **Role:** Simplifies circuit connections.
- **Function:** Provides a platform to connect all the components without soldering. The breadboard allows for easy insertion and removal of components and jumper wires.

6. Jumper Wires (3 male-to-male and 5 male-to-female)

- **Role:** Connect components.

- **Function:** Establish electrical connections between the Arduino, water level sensor, LEDs, and the buzzer.
 - **Male-to-male wires:** Typically used to connect components on the breadboard.
 - **Male-to-female wires:** Used to connect components with pins to the breadboard or Arduino.

Working Mechanism:

1. **Powering the System:** The Arduino is powered on and initializes the pins for input (water level sensor) and output (LEDs and buzzer).
2. **Reading Water Level:** The water level sensor outputs an analog signal that corresponds to the water level. The Arduino reads this analog value through the analog pin (A0).
3. **Processing Data:** The Arduino processes the analog value to determine the water level:
4. Low (0-300)
5. Medium (300-700)
6. High (700-1023)
7. **Indicating Water Level:**
8. If the water level is low, LED 1 is turned on.
9. If the water level is medium, LED 2 is turned on.
10. If the water level is high, LED 3 is turned on, and the buzzer is activated to provide an audible alert.
11. **Updating Output:** The Arduino continuously monitors the water level and updates the LEDs and buzzer accordingly.

SOURCE CODE:

```
const int waterSensorPin = A0; // Analog pin connected to the water level sensor

const int buzzerPin = 8;    // Digital pin connected to the buzzer

const int ledPins[] = {9, 10, 11}; // Digital pins connected to the LEDs


void setup() {
  pinMode(waterSensorPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  for (int i = 0; i < 3; i++) {
```

```
pinMode(ledPins[i], OUTPUT);
}
Serial.begin(9600); // For debugging
}

void loop() {
  int waterLevel = analogRead(waterSensorPin); // Read the water level
  Serial.println(waterLevel); // Print the water level value for debugging

  // Turn on LEDs based on water level
  if (waterLevel < 300) {
    digitalWrite(ledPins[0], HIGH);
    digitalWrite(ledPins[1], LOW);
    digitalWrite(ledPins[2], LOW);
    digitalWrite(buzzerPin, LOW);
  } else if (waterLevel >= 300 && waterLevel < 700) {
    digitalWrite(ledPins[0], HIGH);
    digitalWrite(ledPins[1], HIGH);
    digitalWrite(ledPins[2], LOW);
    digitalWrite(buzzerPin, LOW);
  } else {
    digitalWrite(ledPins[0], HIGH);
    digitalWrite(ledPins[1], HIGH);
    digitalWrite(ledPins[2], HIGH);
    digitalWrite(buzzerPin, HIGH); // Turn on the buzzer if the water level is high
  }

  delay(500); // Wait for half a second before the next reading
}
```

OUTPUT:

Water level: 100

Water level: 150

Water level: 450

Water level: 750

Water level: 900